



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНЖЕНЕРНЫЙ БИЗНЕС И МЕНЕДЖМЕНТ»

КАФЕДРА «ПРОМЫШЛЕННАЯ ЛОГИСТИКА» (ИБМ-3)

Отчет по лабораторной работе № 3

по дисциплине «Технологии машинного обучения»

«Подготовка обучающей и тестовой выборки, кросс-валидация и
подбор гиперпараметров на примере метода ближайших соседей»

Студент ИБМ 3-64Б

(Подпись, дата)

В.А. Леонтьева

Руководитель

(Подпись, дата)

Ю.Е. Гапанюк

Введение

Целью лабораторной работы является изучение методов подготовки данных для машинного обучения, способов разделения выборки на обучающую и тестовую части, а также освоение методов подбора гиперпараметров модели на примере алгоритма ближайших соседей.

В ходе выполнения работы необходимо:

- 1) разделить выборку на обучающую и тестовую;
- 2) обучить модель ближайших соседей;
- 3) оценить качество модели;
- 4) подобрать оптимальный гиперпараметр K ;
- 5) применить кросс-валидацию;
- 6) сравнить результаты исходной и улучшенной моделей.

Для выполнения лабораторной работы использован датасет, содержащий информацию о специалистах различных профессий, уровне образования, опыте работы и заработной плате.

Необходимо решить задачу регрессии, то есть предсказать заработную плату сотрудника на основе входных признаков.

1. Описание датасета

Используемый набор данных содержит 250000 строк и 10 признаков.

Табл.1 - Основные признаки

Признак	Описание
job_title	должность
experience_years	стаж работы
education_level	уровень образования
skills_count	количество навыков
industry	отрасль
company_size	размер компании
location	страна
remote_work	формат работы
certifications	количество сертификатов
salary	заработная плата

Целевая переменная: salary

2. Ход выполнения работы

2.1 Предварительная обработка данных

Перед обучением модели были выполнены следующие действия:

1. Очистка числового признака. В столбце experience_years присутствовали некорректные текстовые значения, которые были преобразованы в пустые значения и затем заменены средним значением столбца.

```
df["experience_years"] = pd.to_numeric(
    df["experience_years"],
    errors="coerce"
)

df["experience_years"].fillna(
    df["experience_years"].mean(),
    inplace=True
)
```

Рис.1 – Очистка числового признака

2. Кодирование категориальных признаков. Текстовые признаки были преобразованы в числовой формат методом Label Encoding.

```
cat_cols = [
    "job_title",
    "education_level",
    "industry",
    "company_size",
    "location",
    "remote_work"
]

le = LabelEncoder()

for col in cat_cols:
    df[col] = le.fit_transform(df[col])
```

Рис.2 – Кодирование признаков

3. Масштабирование данных. Для корректной работы алгоритма ближайших соседей признаки были нормализованы методом StandardScaler.

```
# X и y
X = df.drop("salary", axis=1)
y = df["salary"]

# масштабирование
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

Рис.3 – Масштабирование данных

2.2 Разделение выборки

Данные были разделены методом train_test_split:

80% — обучающая выборка

20% — тестовая выборка

Это позволяет обучить модель на одной части данных и проверить её качество на другой части.

2.3 Обучение исходной модели

В качестве исходной модели использован алгоритм ближайших соседей с параметром: $K = 5$. Модель была обучена на обучающей выборке, после чего выполнен прогноз на тестовой части данных.

Полученные результаты:

```
ИСХОДНАЯ МОДЕЛЬ  
MAE: 17583.04644  
MSE: 488820657.57667196  
R2 : 0.6482959581997518
```

Рис.4 – Метод ближайших соседей

Средняя ошибка прогноза составила около 17,6 тыс. Коэффициент детерминации показывает, что модель объясняет 64.8% зависимости зарплаты от признаков. Это свидетельствует о среднем качестве модели.

2.4 Подбор гиперпараметра методом GridSearchCV

Для поиска оптимального значения параметра K использован метод полного перебора GridSearchCV. Проверялись различные значения количества соседей. Полученные результаты:

```
GRIDSEARCHCV  
Лучший K: {'n_neighbors': 11}  
Лучший score: 0.6575677096926535  
MAE: 17434.778438181816  
MSE: 469866354.242987  
R2 : 0.6619334855600176
```

Рис.5 – Метод GridSearchCV

По сравнению с исходной моделью ошибка MAE уменьшилась, ошибка MSE уменьшилась, коэффициент R^2 увеличился. Это означает, что подбор параметра улучшил качество модели.

2.5 Подбор гиперпараметра методом RandomizedSearchCV

Для дополнительного поиска параметров использовался метод случайного подбора RandomizedSearchCV.

```
RANDOMIZEDSEARCHCV  
Лучший K: {'n_neighbors': 11}  
Лучший score: 0.6575361905030033
```

Рис.6 – Метод RandomizedSearchCV

Метод случайного поиска подтвердил результат полного перебора.

3. Сравнение моделей

Метрика	Исходная модель (K=5)	Оптимальная модель (K=11)
MAE	17583.05	17434.78
MSE	488820657.58	469866354.24
R2	0.6483	0.6619

После подбора гиперпараметров качество модели улучшилось по всем основным метрикам.

Вывод

В ходе выполнения лабораторной работы были изучены методы подготовки выборки и настройки модели машинного обучения.

Были получены практические навыки:

- 1) разделения данных на обучающую и тестовую выборки;
- 2) обучения метода ближайших соседей;
- 3) оценки качества модели;
- 4) использования кросс-валидации;
- 5) автоматического подбора гиперпараметров.

В результате установлено, что оптимальное значение параметра: $K = 11$. Использование подбора параметров позволило повысить качество прогноза заработной платы.

Приложение. Текст программы

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import RandomizedSearchCV
from sklearn.model_selection import KFold
from sklearn.model_selection import ShuffleSplit

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsRegressor

from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

df =
pd.read_csv("/Users/veraleonteva/Desktop/ML/ЛП3/job_salary_prediction_dataset.csv")

# Чистим experience_years
df["experience_years"] = pd.to_numeric(
    df["experience_years"],
    errors="coerce"
)

df["experience_years"].fillna(
    df["experience_years"].mean(),
    inplace=True
)

# Кодирование категорий
cat_cols = [
    "job_title",
    "education_level",
    "industry",
    "company_size",
    "location",
    "remote_work"
]

le = LabelEncoder()

for col in cat_cols:
    df[col] = le.fit_transform(df[col])

# X и y
X = df.drop("salary", axis=1)
y = df["salary"]

# масштабирование
scaler = StandardScaler()
X = scaler.fit_transform(X)

# train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Исходная модель
model = KNeighborsRegressor(n_neighbors=5)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("ИСХОДНАЯ МОДЕЛЬ")
print("MAE:", mean_absolute_error(y_test, pred))
```

```

print("MSE:", mean_squared_error(y_test, pred))
print("R2 :", r2_score(y_test, pred))

# GridSearchCV
params = {"n_neighbors": [1,3,5,7,9,11,13,15]}

cv1 = KFold(n_splits=5, shuffle=True, random_state=42)

grid = GridSearchCV(
    KNeighborsRegressor(),
    params,
    cv=cv1,
    scoring="r2"
)

grid.fit(X_train, y_train)

print("\nGRIDSEARCHCV")
print("Лучший K:", grid.best_params_)
print("Лучший score:", grid.best_score_)

best_model = grid.best_estimator_
pred2 = best_model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, pred2))
print("MSE:", mean_squared_error(y_test, pred2))
print("R2 :", r2_score(y_test, pred2))

# RandomizedSearchCV
cv2 = ShuffleSplit(
    n_splits=5,
    test_size=0.2,
    random_state=42
)

rand = RandomizedSearchCV(
    KNeighborsRegressor(),
    params,
    n_iter=5,
    cv=cv2,
    random_state=42,
    scoring="r2"
)

rand.fit(X_train, y_train)

print("\nRANDOMIZEDSEARCHCV")
print("Лучший K:", rand.best_params_)
print("Лучший score:", rand.best_score_)

```